Trabalho Prático 2
Sistemas Distribuídos

Aluno: Diogo Vilelas
Número: 62347

30 de dezembro de 2025

Conteúdo

1 Introdução **3**

2 Pré-requisitos e Inicialização do Sistema **3**
2.1 Ordem Correta de Inicialização . . . 3
2.2 Inicialização dos Serviços Docker . . . 3
2.3 Resolução de Problemas com Containers Docker Existentes . . . 4
2.4 Inicialização do Servidor Spring Boot . . . 4
2.5 Nota sobre Compilação Inicial do Projeto . . . 5
2.6 Criação dos Dispositivos . . . 5
2.7 Inicialização dos Sensores Simulados . . . 5
2.8 Inicialização do Admin CLI . . . 6

3 Arquitetura do Sistema **6**

4 Configuração da Base de Dados **6**
4.1 Credenciais . . . 6
4.2 Configuração no pgAdmin . . . 7

5 Justificação das Escolhas de Projeto **7**
5.1 Estrutura da Base de Dados . . . 7
5.2 Validação e Política de Armazenamento (Descartar Métricas) . . . 8
5.3 Métodos de Comunicação Cliente–Servidor . . . 8
5.4 Justificação da Utilização de Docker . . . 9
5.5 Escolha do PostgreSQL e pgAdmin . . . 9
5.6 Escolha do ActiveMQ como Broker MQTT . . . 9

6 Sensores Simulados **9**

7 Endpoints Implementados **10**
7.1 POST /api/metrics/ingest . . . 10
7.2 GET /api/metrics/raw . . . 10
7.3 GET /api/metrics/average . . . 10
7.4 GET /api/metrics/average/grouped . . . 11
7.5 GET /api/metrics/all . . . 11

8 Admin CLI **11**

9 Avaliação de Performance **11**
9.1 Metodologia 11
9.2 Consultas SQL Utilizadas 12
9.3 Resultados 12

10 Conclusões **12**

11 Desafios Encontrados **13**

12 Organização do Código e Justificação da Estrutura **14**
12.1 Packages Principais 14
12.2 Integração dos Protocolos 14
12.3 Justificação dos Nomes Utilizados 15

13 Conclusão Final **15**

1 Introdução

Este trabalho tem como objetivo o desenvolvimento de um sistema distribuído para recolha, armazenamento e consulta de métricas ambientais (temperatura e humidade), recorrendo a três protocolos de comunicação distintos: **REST**, **gRPC** e **MQTT**.

O sistema permite:

- Ingestão de métricas provenientes de sensores simulados
- Armazenamento persistente em base de dados PostgreSQL
- Consulta de métricas brutas e agregadas
- Comparação de desempenho entre protocolos

2 Pré-requisitos e Inicialização do Sistema

Esta secção descreve de forma detalhada os passos necessários para colocar todo o sistema em funcionamento, desde os serviços de infraestrutura até aos clientes e ferramentas de administração.

2.1 Ordem Correta de Inicialização

A ordem recomendada para iniciar o sistema é a seguinte:

1. Iniciar os serviços de infraestrutura em Docker
2. Verificar o correto funcionamento dos containers
3. Iniciar o servidor Spring Boot
4. Criar os dispositivos no sistema
5. Iniciar os sensores simulados
6. Iniciar o Admin CLI

Cada um destes passos é detalhado de seguida.

2.2 Inicialização dos Serviços Docker

Os serviços de infraestrutura (PostgreSQL, pgAdmin e ActiveMQ) são executados em containers Docker, definidos no ficheiro `docker-compose.yml`.

Para iniciar os serviços, deve executar-se, na raiz do projeto:

```
docker compose up -d
```

Este comando inicia todos os containers em modo detached. Após a execução, deve confirmar-se que os serviços estão ativos, por exemplo com:

```
docker ps
```

Devem encontrar-se em execução, pelo menos, os containers correspondentes a:

- PostgreSQL
- pgAdmin
- ActiveMQ

2.3 Resolução de Problemas com Containers Docker Existentes

Durante o desenvolvimento do projeto, pode ocorrer um erro ao iniciar os serviços Docker indicando que um ou mais containers já existem. Esta situação é comum quando o projeto é movido de diretoria ou quando os containers foram criados anteriormente com o mesmo nome.

Uma das formas de resolver este problema é parar e remover os containers existentes, bem como os volumes associados, executando:

```
docker compose down -v
```

De seguida, os serviços podem ser iniciados novamente com:

```
docker compose up -d
```

Este procedimento garante que:

- todos os containers são recriados corretamente
- o estado anterior não interfere com a nova execução
- a configuração definida no `docker-compose.yml` é aplicada de forma consistente

Como alternativa, é possível alterar manualmente o campo `container_name` no ficheiro `docker-compose.yml` - tal como indiquei no primeiro projeto desta disciplina.

2.4 Inicialização do Servidor Spring Boot

O servidor central do sistema é uma aplicação Spring Boot responsável por:

- receber métricas via REST, gRPC e MQTT
- validar dispositivos
- persistir dados
- disponibilizar endpoints de consulta

O servidor deve ser iniciado após os serviços Docker estarem ativos.

Para iniciar o servidor, deve executar-se, na diretoria do módulo do servidor:

```
mvn spring-boot:run
```

Após o arranque, o servidor ficará disponível, por defeito, em:

- REST: `http://localhost:8080`
- gRPC: porta configurada no servidor (ex.: 50051)

2.5 Nota sobre Compilação Inicial do Projeto

Durante os testes realizados verifiquei que, na primeira execução do projeto, pode não ser suficiente utilizar apenas os comandos:

```
mvn spring-boot:run
mvn exec:java
```

Em particular, quando o projeto ainda não foi compilado previamente ou quando existem dependências que requerem geração de código (como no caso do gRPC), é necessário forçar explicitamente a fase de compilação.

Assim, na primeira execução, recomenda-se a utilização dos seguintes comandos:

```
mvn clean compile spring-boot:run
```

para o servidor Spring Boot, e:

```
mvn clean compile exec:java
```

para os sensores simulados e para o Admin CLI.

Após esta compilação inicial, as execuções subsequentes podem ser realizadas apenas com `mvn spring-boot:run` e `mvn exec:java`, sem necessidade de recompilar todo o projeto.

2.6 Criação dos Dispositivos

Antes de iniciar qualquer sensor, é **obrigatório criar os dispositivos** no sistema. O servidor apenas aceita métricas provenientes de dispositivos previamente registados e com estado `ACTIVE`.

Os dispositivos podem ser criados de duas formas:

- Diretamente na base de dados PostgreSQL, através de comandos SQL
- Utilizando o Admin CLI, no menu *Gestão de Dispositivos*

Sem esta etapa, todas as métricas recebidas serão descartadas pelo servidor.

2.7 Inicialização dos Sensores Simulados

Após o servidor estar em execução e os dispositivos criados, podem ser iniciados os sensores simulados.

Cada simulador representa um sensor físico e envia métricas periodicamente para o servidor, utilizando um dos protocolos implementados.

Os sensores aceitam parâmetros por linha de comandos:

- **Argumento 1**: identificador do dispositivo
- **Argumento 2**: intervalo de envio em milissegundos

Por exemplo:

```
mvn exec:java -Dexec.args="dev-mqtt01 2000"
```

Caso não sejam fornecidos argumentos, são utilizados valores por defeito (deviceId e intervalo configurados no código - neste caso deviceId vai ser "dev-*protocolo*01"e o tempo de envio será de 2000 ms, por defeito).

2.8 Inicialização do Admin CLI

O cliente de administração (Admin CLI) permite:

- gerir dispositivos
- consultar métricas
- analisar dados agregados
- consultar estatisticas do sistema

O Admin CLI é iniciado executando o respetivo módulo:

```
mvn exec:java
```

Este componente comunica com o servidor REST e deve ser iniciado apenas após o servidor estar operacional.

3 Arquitetura do Sistema

O sistema é composto pelos seguintes componentes:

- Servidor Spring Boot
- Base de Dados PostgreSQL
- ActiveMQ (MQTT)
- pgAdmin
- Clientes simuladores (REST, gRPC, MQTT)
- Admin CLI

4 Configuração da Base de Dados

4.1 Credenciais

PostgreSQL

- Base de dados: `BDT02SD`
- Utilizador: `62347`
- Password: `WORKT02SD`
- Porta: `5432`

pgAdmin

- URL: `http://localhost:5050`
- Email: `162347@alunos.uevora.pt`
- Password: `WORKT02SD`

4.2 Configuração no pgAdmin

Como o pgAdmin está a correr dentro de Docker, a ligação deve ser feita para o nome do serviço PostgreSQL:

- Host name/address: `postgres`
- Port: `5432`
- Maintenance database: `BDT02SD`
- Username: `62347`
- Password: `WORKT02SD`

5 Justificação das Escolhas de Projeto

Esta secção apresenta as principais decisões tomadas ao longo do desenvolvimento, justificando a estrutura da base de dados e os métodos de comunicação entre clientes e servidor, conforme solicitado no enunciado.

5.1 Estrutura da Base de Dados

Foi adotado um modelo relacional simples, com duas tabelas principais: `devices` e `metrics`. A opção por separar dispositivos de métricas permite: (i) manter informação estática e administrativa (dispositivos) separada dos dados temporais (métricas), (ii) garantir validação e controlo centralizado do sistema, e (iii) suportar consultas agregadas por sala/departamento/piso/edifício sem duplicação de informação.

Tabela `devices` A tabela `devices` armazena os dispositivos registados no sistema e a respetiva meta-informação, necessária para:

- validar se um sensor pode ou não enviar dados (`status=ACTIVE`);
- permitir consultas por entidades lógicas (sala, departamento, piso e edifício);
- suportar múltiplos protocolos para diferentes dispositivos (campo `protocol`).

Campos relevantes e justificação:

- `id`: chave primária e identificador usado em todos os protocolos;
- `protocol`: identifica o protocolo associado ao dispositivo (MQTT/gRPC/REST);
- `building`, `floor`, `department`, `room`: permitem agregação e filtragem conforme exigido no enunciado;
- `status`: permite ativar/desativar sensores sem alterar código (controlo administrativo).

Tabela `metrics` A tabela `metrics` armazena as observações enviadas pelos sensores. Foi escolhido um formato orientado a séries temporais, com `timestamp` e atributos numéricos (`temperature`, `humidity`). Cada métrica referencia o dispositivo através de `device_id`, permitindo associar rapidamente as observações ao contexto (sala/departamento/piso/edifício) via `devices`.

Campos relevantes e justificação:

- `id`: chave primária auto-gerada (facilita indexação e ordenação);
- `device_id`: associa a métrica ao dispositivo que a originou;
- `temperature`, `humidity`: variáveis medidas no trabalho;
- `timestamp`: necessário para filtros temporais e para calcular médias num intervalo.

5.2 Validação e Política de Armazenamento (Descartar Métricas)

Uma decisão relevante foi implementar a validação centralizada no servidor: **apenas métricas de dispositivos existentes e com estado `ACTIVE` são armazenadas**.

Esta escolha foi feita para:

- garantir integridade dos dados (evitar *noise* de sensores desconhecidos);
- permitir controlo administrativo (ativar/desativar sensores sem alterar clientes);
- cumprir o comportamento esperado num cenário real, onde sensores podem ser retirados de serviço.

5.3 Métodos de Comunicação Cliente–Servidor

Foram implementados três métodos de comunicação distintos, com o objetivo de comparar desempenho e adequação a diferentes cenários, conforme exigido no enunciado.

REST (HTTP + JSON) O protocolo REST foi utilizado por ser simples, amplamente suportado e ideal para integração e testes rápidos. A ingestão REST é feita via `POST /api/metrics/ingest`, transportando JSON. A escolha de REST facilita também a implementação do Admin CLI.

gRPC (HTTP/2 + Protobuf) O gRPC foi escolhido como alternativa mais eficiente e estruturada, utilizando serialização binária (Protocol Buffers) e HTTP/2. Esta abordagem reduz overhead face ao JSON e permite contratos bem definidos entre cliente e servidor, sendo adequada para comunicação entre serviços em sistemas distribuídos.

MQTT (Publish/Subscribe) O MQTT foi escolhido por ser um protocolo leve, adequado a cenários IoT e sensores com envio frequente. A comunicação é feita no modelo publish/subscribe, com os sensores a publicar no tópico configurado (`sdist/metrics`) e o servidor a subscrever e processar mensagens. Esta abordagem desacopla produtores (sensores) do consumidor (servidor) e é eficiente sob carga elevada, como evidenciado na análise de desempenho.

5.4 Justificação da Utilização de Docker

A utilização de Docker (PostgreSQL, pgAdmin e ActiveMQ) foi escolhida para:

- simplificar configuração e execução em diferentes máquinas;
- reduzir dependências locais e conflitos de versões.

5.5 Escolha do PostgreSQL e pgAdmin

O sistema utiliza PostgreSQL como sistema de gestão de base de dados relacional devido à sua robustez, desempenho e ampla utilização em contextos académicos e profissionais. O PostgreSQL oferece suporte eficiente a operações de agregação, filtros temporais e integridade referencial, requisitos centrais para o armazenamento e consulta de métricas ambientais.

A utilização do **pgAdmin** como ferramenta de administração foi uma escolha complementar, com o objetivo de:

- facilitar a visualização e inspeção direta dos dados armazenados;
- permitir validação manual dos resultados das consultas SQL utilizadas na avaliação de desempenho;
- ajuda durante o desenvolvimento devido ao facto de podermos confirmar os dados que temos nas tabelas e comparar com as respostas obtidas / operações realizadas.
- tornar o sistema mais acessível do ponto de vista operacional e pedagógico.

Desta forma, a combinação PostgreSQL + pgAdmin contribuiu tanto para a robustez técnica do sistema como para a clareza na análise e validação dos dados.

5.6 Escolha do ActiveMQ como Broker MQTT

O broker MQTT utilizado no sistema foi o **ActiveMQ**. Esta escolha foi feita deivo a ter sido a solução apresentada nas aulas práticas da unidade curricular.

A utilização do ActiveMQ permitiu implementar de forma consistente o modelo publish/subscribe exigido pelo protocolo MQTT, assegurando uma ingestão eficiente de métricas provenientes dos sensores e possibilitando a avaliação do seu desempenho em cenários de carga elevada.

6 Sensores Simulados

Todos os sensores aceitam parâmetros por linha de comandos:

- **Argumento 1**: deviceId
- **Argumento 2**: intervalo de envio em milissegundos

Exemplos MQTT:

```
mvn exec:java -Dexec.args="dev-mqtt01 2000"
```

gRPC:

```
mvn exec:java -Dexec.args="dev-grpc01 2000"
```

REST:

```
mvn exec:java -Dexec.args="dev-rest01 2000"
```

NOTA: Caso não se passe argumento e apenas se faça:

```
mvn exec:java
```

Os simuladores vão usar os dev-*protocolo*01 com 2000 milissegundos de tempo de envio como default.

7 Endpoints Implementados

Foram implementados todos os endpoints exigidos no enunciado, bem como alguns adicionais para apoio ao sistema e ao Admin CLI.

7.1 POST /api/metrics/ingest

Este endpoint é responsável pela ingestão de métricas provenientes dos sensores REST.

Recebe um objeto JSON com:

- deviceId
- temperature
- humidity
- timestamp

O servidor valida se o dispositivo existe e se está ativo. Caso contrário, a métrica é descartada.

7.2 GET /api/metrics/raw

Este endpoint devolve métricas brutas de um dispositivo específico, permitindo filtrar por intervalo temporal (`from` e `to`). É utilizado principalmente pelo Admin CLI para obter valores individuais e métricas mais recentes.

7.3 GET /api/metrics/average

Este endpoint devolve a média da temperatura e humidade para uma entidade específica, conforme definido no enunciado.

Permite agregação por:

- Sala

- Departamento
- Piso
- Edifício

Internamente, o servidor resolve os dispositivos associados à entidade e calcula a média das métricas correspondentes no intervalo temporal indicado (ou últimas 24 horas por defeito).

7.4 GET /api/metrics/average/grouped

Endpoint adicional que devolve médias agrupadas por todas as entidades de um determinado nível (ex.: todas as salas ou todos os departamentos). É útil para dashboards e análises globais.

7.5 GET /api/metrics/all

Endpoint adicional que devolve todas as métricas armazenadas, com possibilidade de filtragem temporal. É utilizado pelo Admin CLI para visualização tabular completa.

8 Admin CLI

O Admin CLI permite:

- Gestão de dispositivos
- Consulta por sala, departamento, piso e edifício
- Definição de intervalos temporais
- Visualização tabular

9 Avaliação de Performance

9.1 Metodologia

Foram definidos dois cenários de teste:

- **Carga A (leve)**: dois sensores ativos com intervalo de 2 segundos
- **Carga B (elevada)**: dois sensores a enviar métricas continuamente (de 20 em 20 ms)

Antes de cada teste, a tabela de métricas foi limpa para garantir resultados independentes:

```
TRUNCATE TABLE metrics;
```

9.2 Consultas SQL Utilizadas

(A) Total de mensagens, duração real e mensagens por segundo

```
SELECT
  count(*) AS total,
  min(timestamp) AS first_ts,
  max(timestamp) AS last_ts,
  round(
    (count(*) / nullif(extract(epoch from (max(timestamp) - min(
       timestamp))), 0))::numeric,
    3
  ) AS msgs_per_sec
FROM metrics;
```

(B) Verificação de perdas por dispositivo

```
SELECT device_id, count(*) AS total
FROM metrics
GROUP BY device_id
ORDER BY total DESC;
```

Estas consultas permitiram avaliar:

- Throughput efetivo
- Distribuição de carga
- Perdas de mensagens por protocolo

9.3 Resultados

Carga A (Leve)

Protocolo	Total	Msgs/s	Perdas
MQTT	28	1.03	Não
gRPC	28	1.01	Não
REST	28	1.02	Não

Carga B (Elevada)

Protocolo	Total	Msgs/s	Perdas
MQTT	2449	87.45	Menores
gRPC	1875	67.92	Médias
REST	1848	67.13	Maiores

10 Conclusões

Em cargas reduzidas, os três protocolos (MQTT, gRPC e REST) apresentaram resultados semelhantes, com taxas de ingestão equivalentes e sem perdas de mensagens.

No entanto, ao aumentar a carga (intervalo de 20 ms), observaram-se diferenças significativas. O protocolo MQTT apresentou o maior throughput, com cerca de 87 mensagens por segundo, superando gRPC e REST, que ficaram próximos dos 67 mensagens por segundo.

Estes resultados indicam que o modelo publish/subscribe e o menor overhead do MQTT o tornam mais adequado para cenários de ingestão contínua de dados em grande volume. O gRPC apresenta um desempenho intermédio, enquanto o REST revela maiores limitações sob carga elevada devido ao overhead do protocolo HTTP e do formato JSON.

Com base nos resultados obtidos, conclui-se que os três protocolos analisados apresentam características distintas, adequadas a diferentes cenários de utilização.

- O **MQTT** revelou-se o protocolo mais eficiente em cenários de elevada carga, apresentando maior taxa de mensagens por segundo e menor perda de dados, o que o torna particularmente adequado para sistemas IoT e aplicações com grande número de dispositivos sensores.

- O **gRPC** demonstrou um desempenho intermédio, combinando eficiência com uma estrutura fortemente tipada e comunicação binária, sendo uma solução adequada para sistemas distribuídos que exigem desempenho consistente e contratos bem definidos entre cliente e servidor.

- O **REST**, apesar de apresentar maior overhead e menor escalabilidade em cargas elevadas, continua a ser uma solução robusta, simples e amplamente suportada, sendo mais indicada para operações administrativas, configuração e integração com sistemas externos.

11 Desafios Encontrados

Ao longo do desenvolvimento do projeto, encontrei alguns desafios técnicos, nomeadamente:

- A definição de uma **metodologia adequada para avaliação de desempenho**, em particular a escolha de métricas relevantes (throughput, duração real dos testes e perdas por dispositivo) e a forma de garantir comparações justas entre os diferentes protocolos.

- A correta **gestão de dependências e plugins Maven**, sobretudo no que diz respeito à integração de gRPC, MQTT e geração de código a partir de ficheiros `.proto`, onde pequenos erros de configuração nos `pom.xml` originaram falhas de compilação e execução.

- Entender o funcionamento dos três protocolos distintos utilizados e como os implementar.

- Decidir como implementar cada cliente simulador em termos de se eles deviam ter acesso ou não a atualizacoes na base de dados, mas acabei por decidir que não fazia sentido, pois apenas servem para simular um sensor a enviar dados para o servidor, o servidor é que resolve e sabe se os dados são validos e se o sensor esta ativo ou não (ou até se existe ou não).

12 Organização do Código e Justificação da Estrutura

A estrutura do projeto foi organizada seguindo boas práticas comuns em aplicações Spring Boot, com separação clara de responsabilidades (*separation of concerns*). Esta abordagem facilita a manutenção do código, reduz acoplamento entre componentes e melhora a escalabilidade do sistema.

12.1 Packages Principais

`controller` Contém as classes responsáveis por expor a API HTTP (endpoints REST). Cada controlador define rotas e valida os parâmetros de entrada, delegando a lógica de negócio para as camadas inferiores. Exemplo: `MetricController` implementa os endpoints de ingestão e consulta definidos no enunciado.

`service` Contém a lógica de negócio do sistema. Esta camada centraliza regras e validações, evitando que o controlador tenha comportamento complexo. Exemplo: `MetricIngestService` decide se uma métrica deve ser armazenada (verifica existência e estado do dispositivo), garantindo que regras do sistema não ficam espalhadas por múltiplas classes.

`repository` Inclui interfaces de acesso a dados, usando Spring Data JPA. Esta camada abstrai completamente o acesso à base de dados, permitindo que o restante sistema não dependa de SQL explícito nem de detalhes de persistência. Exemplos: `DeviceRepository` e `MetricRepository` fornecem métodos de consulta e agregação (médias, filtros por intervalo, etc.).

`model` Contém as entidades persistentes (mapeadas para tabelas na base de dados). Estas classes representam a estrutura de dados do sistema e são utilizadas pelo JPA para gerar e atualizar o esquema da base de dados. Exemplos: `Device` representa os dispositivos registados, e `Metric` representa cada observação enviada por sensores.

`dto` Inclui classes de transporte de dados (*Data Transfer Objects*). Estas classes existem para separar o formato exposto nas APIs e nos clientes da estrutura interna das entidades. Esta decisão evita expor diretamente entidades JPA para o exterior, reduz riscos de acoplamento e facilita a validação e evolução da API. Exemplos: `MetricIngestRequest` representa o corpo recebido no endpoint `/api/metrics/ingest`; `AverageMetricsResponse` representa a resposta de agregação devolvida no endpoint `/api/metrics/average`.

12.2 Integração dos Protocolos

`grpc` Agrupa as classes relacionadas com o servidor gRPC e a integração com o código gerado a partir do ficheiro `.proto`. A separação em package próprio evita misturar lógica gRPC com a lógica HTTP/REST e permite evoluir o serviço gRPC de forma independente.

`mqtt` Agrupa o componente responsável por subscrever o tópico MQTT e processar as mensagens recebidas. Tal como no caso do gRPC, a separação em package próprio facilita a modularidade e evita dependências desnecessárias na camada REST.

12.3 Justificação dos Nomes Utilizados

Os nomes atribuídos aos packages (`controller`, `service`, `repository`, `model`, `dto`) seguem convenções amplamente reconhecidas em projetos Spring Boot e facilitam a leitura imediata do projeto por terceiros. Além disso, cada classe foi colocada no package correspondente à sua responsabilidade principal, promovendo organização, reutilização e clareza na arquitetura.

13 Conclusão Final

O trabalho desenvolvido permitiu-me implementar de um sistema distribuído completo para ingestão e consulta de métricas, integrando múltiplos protocolos de comunicação e componentes tecnológicas distintas.

Penso que cumpri todos os requisitos definidos no enunciado, desde a ingestão de dados por REST, gRPC e MQTT, até à persistência em base de dados, disponibilização de endpoints REST e desenvolvimento de um cliente de administração.

A análise de desempenho efetuada evidenciou diferenças claras entre os protocolos, permitindo compreender o impacto do modelo de comunicação, serialização de dados e overhead de cada abordagem. Este trabalho contribuiu, assim, para uma melhor compreensão prática dos desafios e compromissos inerentes ao desenvolvimento de sistemas distribuídos modernos.